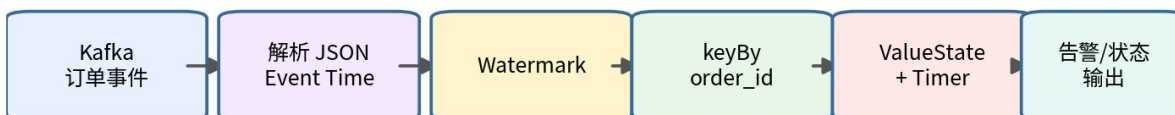


大数据综合项目实战

第 8 讲：订单 15 分钟未支付自动告警

Kafka → Event Time → keyBy(order_id) → ValueState → Event-Time Timer → Timeout Alert

第8讲实时状态机：订单 CREATE 后 15 分钟仍未支付则告警



本讲核心：第 7 讲解决“按时间窗口统计”；第 8 讲解决“每个订单都要记住自己的历史，并在未来某个时间点检查它”。这是 Window 之外最典型的有状态实时业务。

1. 学完这一讲，你必须真正会什么

- 理解为什么“订单 15 分钟未支付告警”不能简单用一个 1 分钟 Window 解决。
- 理解 keyBy(order_id) 后，每个订单可以拥有独立的 Keyed State 和 Timer。
- 能解释 ValueState 保存什么、什么时候创建、什么时候清理。
- 能解释 registerEventTimeTimer(deadline) 注册的 not “睡眠 15 分钟”，而是等待 Watermark 到达某个业务时间。
- 能正确处理 CREATE → PAY、CREATE → CANCEL、CREATE → 超时三条状态路径。
- 知道为什么 PAY/CANCEL 到达后必须 deleteEventTimeTimer(), 否则会产生假告警。
- 理解 Event-Time Timer 与 Processing-Time Timer 的业务语义差异。
- 能运行 Kafka + Flink 作业，手工发送演示事件，并用标准答案验收输出。
- 能解释 State/Timer 为什么可以和 Checkpoint 一起恢复。

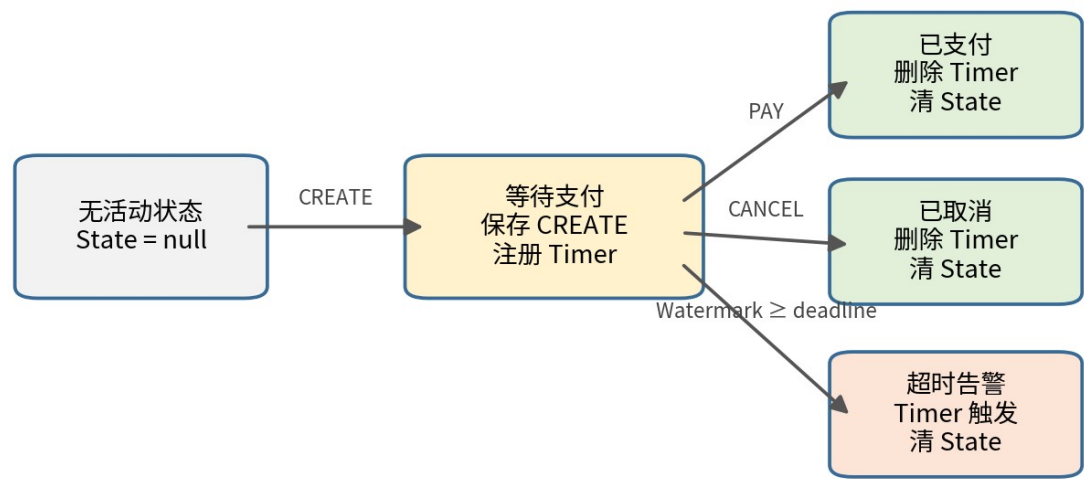
最低闭环：Topic 有数据 → Job RUNNING → CREATE 进入 State → Timer 注册 → PAY/CANCEL 能删除 Timer → Watermark 推进 → 真正未支付订单产生 TIMEOUT_ALERT → 结果与手工状态机一致。

2. 业务需求：这次不统计“多少”，而是追踪“每一笔订单后来怎么样”

需求：订单 CREATE 后，如果 15 分钟内收到 PAY，就认为按时支付；如果收到 CANCEL，就取消等待；如果超过 15 分钟仍然没有 PAY/CANCEL，就输出超时告警。

事件序列	系统应该做什么	是否告警
CREATE → PAY (15 分钟内)	删除 Timer，清理 State，输出 PAID_IN_TIME	否
CREATE → CANCEL	删除 Timer，清理 State，输出 CANCELLED	否
CREATE → 15 分钟仍无后续	Timer 触发，输出 TIMEOUT_ALERT，清 State	是
PAY 但没有活动 CREATE	无法匹配活动订单，输出诊断状态	否（但需排查数据）

每个 order_id 都有自己的一份状态机



这类问题的本质不是“每分钟统计一次”，而是一个长期存在的状态机：每个 order_id 都处在不同状态，而且每个订单的截止时间也不同。

3. 为什么普通 Window 不够用

Window 擅长解决“在某段时间内聚合一批事件”，例如每分钟销售额、每 5 分钟访问量。订单超时则是“每个 Key 都有自己独立的截止时间”。O3001 10:00 创建，截止 10:15；O3002 10:01 创建，截止 10:16；O3004 10:03 创建，截止 10:18。

判断方法：如果问题是“固定时间桶里算一批数据”，优先 Window；如果问题是“每个 Key 记住历史，并在自己的未来时刻做动作”，优先 State + Timer / ProcessFunction。

4. 本讲演示数据与标准答案

演示数据的“标准答案”

订单	CREATE	后续事件	预期输出	意义
O3001	CREATE 10:00	PAY 10:05	PAID_IN_TIME	不告警
O3002	CREATE 10:01	无	TIMEOUT_ALERT 10:16	告警
O3003	CREATE 10:02	CANCEL 10:10	CANCELLED	不告警
O3004	CREATE 10:03	无	TIMEOUT_ALERT 10:18	告警
O3999	CREATE 10:19	PAY 10:19:10	PAID_IN_TIME	辅助推进 Watermark

演示数据故意设计为 5 个订单角色：O3001 正常支付、O3002 超时、O3003 取消、O3004 超时、O3999 既负责验证正常支付，又负责把 Event Time 推进到 10:19 以后。

标准答案：集合必须包含：O3001=PAID_IN_TIME，O3002=TIMEOUT_ALERT，O3003=CANCELLED，O3004=TIMEOUT_ALERT，O3999=PAID_IN_TIME。输出顺序可能不同，但业务集合必须等价。

5. Timer 的最容易误解之处：15 分钟有两种完全不同的含义

同样是“15 分钟”，语义可以完全不同

Event-Time Timer
registerEventTimeTimer(deadline)
由 Watermark 推进触发
适合可重放、按业务时间计算

Processing-Time Timer
registerProcessingTimeTimer(now+15m)
由机器时钟触发
适合真实墙钟 SLA

本讲使用 Event-Time Timer 来训练 State + Watermark + Timer 的组合；
真实“用户必须 15 分钟内支付”的告警是否应该用 Processing Time，要由业务 SLA 决定。

Event-Time Timer 的“15 分钟”是业务事件时间。例如 CREATE.event_time=10:01:00，则 deadline=10:16:00。它什么时候真正触发，取决于 Watermark 是否推进到 10:16。

Processing-Time Timer 的“15 分钟”是机器当前时间。例如系统在 15:03:20 收到 CREATE，就注册到 15:18:20，基本不关心事件自己的 event_time。

现实选择：如果目标是“无论有没有新事件，用户真实等待 15 分钟后就必须收到告警”，Processing Time 往往更符合墙钟 SLA；如果目标强调按业务事件时间计算、离线重放仍得到一致结果，则 Event Time 更自然。本讲选择 Event-Time Timer，是为了把 Watermark、State、Timer 真正串起来。

6. 先看一个正常支付订单：为什么 Timer 不会误报警



关键：Timer 是否触发由 Watermark/Processing Time 决定；不是“注册后固定睡 15 分钟”。

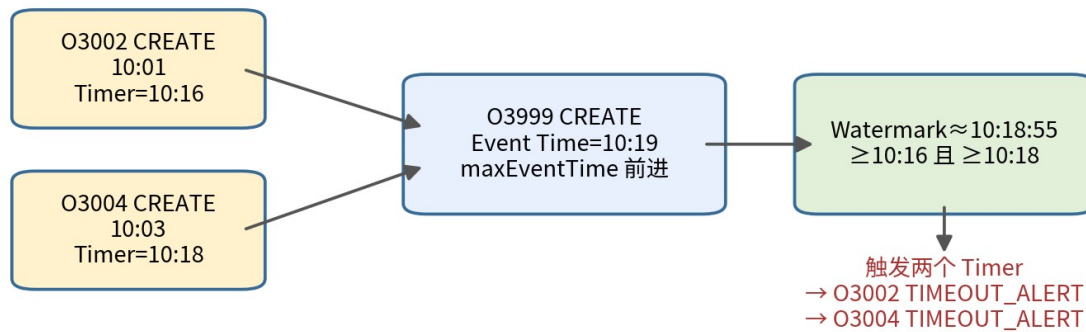
O3001 在 10:00 CREATE，系统注册 10:15 的 Event-Time Timer；10:05 PAY 到达后，必须执行 deleteEventTimeTimer(10:15)，然后清除订单 State。以后 Watermark 即使走到 10:15、10:20，也不应再产生 O3001 超时告警。

常见 Bug：只 clear State、不删除 Timer 虽然 onTimer 中可能因为 State=null 而不输出，但会保留无意义 Timer；只删 Timer、不清 State 会造成状态泄漏。正确做法是“Timer 和 State 成对维护”。

7. 为什么 O3002 / O3004 能在几秒钟实验里模拟“15 分钟超时”

我们不会真的坐在电脑前等 15 分钟。演示数据直接把业务 Event Time 从 10:00 推进到 10:19。Event-Time Timer 关心的是 Watermark，而不是现实墙钟经过了多久。

Event-Time Timer 依赖 Watermark 前进



O3999 的 CREATE 时间是 10:19:00。最大乱序设置为 5 秒，因此 Watermark 很快可以推进到约 10:18:55：这已经超过 O3002 的 10:16 deadline 和 O3004 的 10:18 deadline，于是两个 Timer 都会触发。

很关键：如果 Kafka 后面再也没有任何更晚 Event Time 的事件，Watermark 可能不再前进，Event-Time Timer 也可能一直不触发。这正是 Event Time 与墙钟 Timer 的根本区别。

8. 上机前固定目录与版本策略

第 8 讲与第 7 讲保持同一技术线：Flink 1.20.3、Kafka Connector 3.4.0-1.20、Java 11。建议代码目录：

```
~/bigdata-project/lecture8-order-timeout-alert/
```

版本适配：如果你的 VM 中 Flink 版本不同，请先执行 `flink --version`，再调整 `pom.xml`。不要为了 Flink 永久替换 Hadoop 的 Java 8；可在当前终端临时切到 Java 11/17。

9. 第一步：启动前检查（每次课都做）

启动 Ubuntu 虚拟机 → Windows 打开 VS Code → Remote SSH → 打开远程目录 → Terminal。依次执行：

```
hostname
pwd
whoami
java -version
mvn -version
echo $KAFKA_HOME
echo $FLINK_HOME
ss -lntp | grep 9092 || true
$FLINK_HOME/bin/flink --version
```

然后确认 Flink Web UI：

```
http://<虚拟机 IP>:8081
```

通过标准：Kafka 9092 正常监听；Flink Web UI 可访问；至少 1 个 TaskManager；Available Task Slots > 0；Maven 与 Java 可用。

10. 第二步：创建干净的教学 Topic

为了避免第 6/7 讲旧消息干扰，本讲单独使用 orders_timeout_l8。第一次手工执行：

```
$KAFKA_HOME/bin/kafka-topics.sh --bootstrap-server localhost:9092 \  
--delete --topic orders_timeout_l8 || true  
  
sleep 2  
  
$KAFKA_HOME/bin/kafka-topics.sh --bootstrap-server localhost:9092 \  
--create --if-not-exists --topic orders_timeout_l8 \  
--partitions 3 --replication-factor 1  
  
$KAFKA_HOME/bin/kafka-topics.sh --bootstrap-server localhost:9092 \  
--describe --topic orders_timeout_l8
```

11. 第三步：项目目录里每个文件是干什么的

```
lecture8-order-timeout-alert/  
├── pom.xml  
├── src/main/java/edu/project/realtime/  
│   └── OrderTimeoutAlertJob.java  
├── data/  
│   ├── order_timeout_demo.jsonl  
│   └── late_pay_after_timeout.jsonl  
├── expected_results.txt  
├── 08_emit_keyed_events.py  
├── 08_reset_topic.sh  
├── 08_send_demo_events.sh  
├── 08_send_late_pay.sh  
└── 08_build_and_run.sh
```

第一次不要直接执行一键脚本。先把 Java 主程序中的 Source、Watermark、keyBy、ValueState、Timer 和输出一段段看明白。

12. 第四步：Kafka Source 与 Watermark - 和第 7 讲相同，但作用变了

```
WatermarkStrategy<OrderEvent> wm = WatermarkStrategy  
    .<OrderEvent>forBoundedOutOfOrderness(Duration.ofSeconds(5))  
    .withTimestampAssigner((e, ts) -> e.eventTimeMillis)  
    .withIdleness(Duration.ofSeconds(20));  
  
DataStream<OrderEvent> withWatermarks = parsed  
    .assignTimestampsAndWatermarks(wm);
```

第 7 讲 Watermark 用来关闭 Window；第 8 讲 Watermark 用来触发 Event-Time Timer。底层仍是同一条“事件时间进度线”，只是下游组件对它的响应不同。

13. 第五步：为什么必须 keyBy(order_id)

```
DataStream<MonitorResult> results = withWatermarks
    .keyBy(e -> e.orderId)
    .process(new OrderTimeoutProcess());
```

Keyed State 和 Keyed Timer 都依赖“当前 Key”。O3001 的 State、Timer 与 O3002 完全隔离；当 processElement() 正在处理 O3001 时，state.value() 看到的是 O3001 自己的状态。

不要用 city 做 Key：这次要追踪的是每一笔订单。如果 keyBy(city)，同一城市的多个订单会共享当前 Key 的状态命名空间，业务语义就错了。

14. 第六步：ValueState 保存什么

```
private transient ValueState<OrderState> orderState;

public void open(Configuration parameters) {
    ValueStateDescriptor<OrderState> desc =
        new ValueStateDescriptor<>("order-state", OrderState.class);
    orderState = getRuntimeContext().getState(desc);
}
```

OrderState 中保存：orderId、userId、city、amount、CREATE 时间、timerTimestamp。State 的目标不是保存所有历史事件，而是保存“以后判断超时所需的最小信息”。

15. 第七步：CREATE - 保存状态并注册 Event-Time Timer

```
if ("CREATE".equals(e.eventType)) {
    if (current == null) {
        long deadline = e.eventTimeMillis + TIMEOUT_MILLIS;
        ...
        orderState.update(s);
        ctx.timerService().registerEventTimeTimer(deadline);
    }
    return;
}
```

本讲采取一个简化但很重要的幂等策略：同一 order_id 的第二次 CREATE 不重新延长截止时间。否则上游重复消息可能把“15 分钟超时”不断往后推。

16. 第八步：PAY / CANCEL - 删除 Timer，再清 State

```
ctx.timerService().deleteEventTimeTimer(current.timerTimestamp);
out.collect(...);
orderState.clear();
```

PAY 和 CANCEL 的共同点是：这笔订单不应该继续等待。因此先删除未来 Timer，再清理 Keyed State。这样既不会误告警，也不会留下长期无用状态。

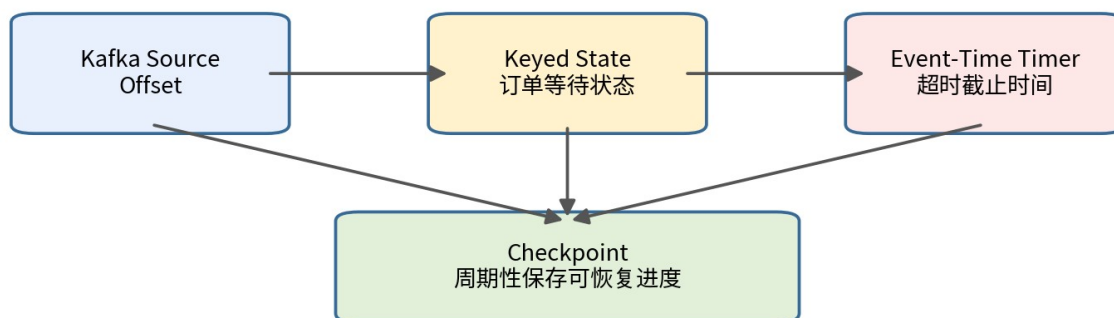
17. 第九步：onTimer - 真正产生超时告警

```
public void onTimer(long timestamp,
                    OnTimerContext ctx,
                    Collector<MonitorResult> out) throws Exception {
    OrderState current = orderState.value();
    if (current != null && current.timerTimestamp == timestamp) {
        out.collect(MonitorResult.fromState(
            "TIMEOUT_ALERT", current, timestamp,
            "Watermark 已推进到超时点，订单仍未 PAY/CANCEL"));
        orderState.clear();
    }
}
```

onTimer() 触发时再次检查 State，是非常重要的防御式思维：只有当前订单仍然处于活动等待状态，而且这个 Timer 正好是记录中的 deadline，才输出告警。

18. 第十步：Checkpoint 为什么和本讲关系很大

故障恢复时：Offset + State + Timer 必须回到一致进度



代码中启用了每 5 秒一次 Checkpoint。Flink 管理的 Keyed State 和 Timer 都属于可恢复状态；Kafka Source 的消费进度也参与 Checkpoint。故障后如果只恢复 Offset、不恢复 Timer，或只恢复 Timer、不恢复 State，都可能出现漏告警或重复告警。

边界：本讲 stdout print 只是教学 Sink，不具备生产级事务 Exactly-once。这里重点训练的是 Source Offset + State + Timer 的一致恢复思想。

19. 第十一步：编译并提交 Job

```
cd ~/bigdata-project/lecture8-order-timeout-alert
mvn -DskipTests clean package

$FLINK_HOME/bin/flink run -d \
  -c edu.project.realtime.OrderTimeoutAlertJob \
  target/flink-order-timeout-alert-1.0-SNAPSHOT-all.jar

$FLINK_HOME/bin/flink list -r
```

Web UI 应看到：`BigData Project L8 - Order Timeout Alert` 为 RUNNING。然后找到 TaskManager stdout：

```
TM_OUT=$(ls -t "$FLINK_HOME"/log/*taskexecutor*.out 2>/dev/null | head -1)
echo "$TM_OUT"
tail -f "$TM_OUT"
```

20. 第十二步：发送 8 条演示事件

先打开 `data/order\_timeout\_demo.jsonl`，对照 expected_results.txt 手算每个订单的最终状态。然后执行：

```
./08_send_demo_events.sh
```

标准输出集合应包含（顺序可能不同）：

```
ORDER_MONITOR> status=PAID_IN_TIME, order_id=03001, ...
ORDER_MONITOR> status=CANCELLED, order_id=03003, ...
ORDER_MONITOR> status=TIMEOUT_ALERT, order_id=03002, ... action_time=2026-09-29 10:16:00
ORDER_MONITOR> status=TIMEOUT_ALERT, order_id=03004, ... action_time=2026-09-29 10:18:00
ORDER_MONITOR> status=PAID_IN_TIME, order_id=03999, ...
```

验收重点： O3001 和 O3003 绝对不能出现 TIMEOUT_ALERT；O3002 和 O3004 必须各出现一次 TIMEOUT_ALERT；O3999 的作用是推进时间并验证正常支付。

21. 第十三步：为什么 O3999 是这个实验的关键

如果数据只发到 O3003 的 CANCEL(10:10)，最大 Event Time 仍然只有 10:10 左右，Watermark 不可能自动跳到 10:18。O3002/O3004 的 Event-Time Timer 就不会触发。O3999 的 10:19 CREATE 是“时间推进器”，让实验几秒钟内模拟出业务时间已经过去 19 分钟。

不要形成错误认识： 生产系统不需要人为发送 O3999。真实业务流通常持续有事件，Event Time 自然前进；如果业务流长期完全空闲而告警仍必须按墙钟准时发生，应重新考虑 Processing-Time Timer 是否更合适。

22. 第十四步：超时后再来 PAY，会发生什么

确认 O3004 已经产生 TIMEOUT_ALERT 后，再执行：

```
./08_send_late_pay.sh
```

因为超时 Timer 已经触发并清除了 O3004 State，所以这条 PAY 找不到活动 CREATE，程序会输出：

```
status=PAY_WITHOUT_ACTIVE_CREATE, order_id=03004, ...
```

这说明“告警已经发出”与“后来又发生支付”是两个业务事实。生产系统通常需要下游告警系统支持告警关闭/恢复状态，而不是假设一条晚到 PAY 能把已经发出的告警从历史里删除。

23. 第十五步：Web UI 与证据链怎么验收

第8讲验收不是“看到一条告警”，而是一条完整证据链



层次	必须看到的证据	若没有
Kafka	orders_timeout_l8 有消息	先查 Broker/Topic/Producer
Runtime	Job RUNNING、TaskManager 有 records in	查 JAR/Connector/异常日志
State/Timer	CREATE 后没有立即输出超时；PAY/CANCEL 能改变路径	查 keyBy 与 ProcessFunction
Event Time	O3999 后超时 Timer 被触发	查 Watermark/时间戳
业务结果	两条超时、两条正常终止、一条辅助正常支付	查状态机逻辑
Checkpoint	Completed Checkpoints 持续增加	查 Checkpoint 配置/存储/反压

24. 最常见的 9 类错误与固定排查顺序

现象	优先检查	常见原因
CREATE 后立刻告警	Timer 类型/时间戳	把 Event Time 当 Processing Time 或单位错
永远不告警	Watermark 是否推进	没有更晚事件、时间戳解析错
已支付仍告警	PAY 是否匹配同 order_id	Key 不一致或没有 deleteTimer
取消后仍告警	CANCEL 分支	未删 Timer/未清 State
多个订单互相干扰	keyBy 字段	错误使用 city/user_id
PAY_WITHOUT_ACTIVE_CREATE 很多	上游数据顺序/重复/超时已清理	事件缺失或迟到
Job 重启后行为异常	Checkpoint	恢复点不一致或未成功完成
Kafka 有数据但 records in=0	Topic/Group/Connector	连接或依赖问题
Web UI RUNNING 但答案不对	expected_results.txt	RUNNING 只代表作业活着

固定排查顺序：环境 → Kafka → JSON/Event Time → keyBy → State → Timer → Watermark → Checkpoint → 业务答案。

25. 练习题

1. 为什么订单超时更适合 State + Timer，而不是一个统一的 15 分钟滚动窗口？
2. 如果 O3001 PAY 到达后只 clear State、不 deleteEventTimeTimer，会有什么工程问题？
3. 如果业务要求“现实世界真的过 15 分钟就必须告警，即使 Kafka 完全没有新事件”，Event-Time Timer 还是最佳选择吗？
4. 为什么本讲必须 keyBy(order_id)，而不是 keyBy(city)？
5. Checkpoint 对 State、Timer、Kafka Offset 的一致恢复为什么重要？
6. O3004 超时告警已经发出后再来 PAY，为什么不能简单“撤回历史告警”？

26. 参考答案

1. 因为每个订单的截止时间不同，且需要记住每个 Key 的历史状态；Window 主要面向固定时间桶内的一组事件。
2. 通常不会产生业务误告警，因为 onTimer 还能看到 State 已空，但会留下无意义 Timer，增加运行时管理开销；最佳实践是 Timer 和 State 成对清理。
3. 不一定。若必须按墙钟 SLA 准时触发，Processing-Time Timer 往往更直接；Event Time 更适合业务时间语义与可重放一致性。
4. 因为状态机的实体是订单。city 下面可能有很多订单，使用 city 会把不同订单放进同一个 Keyed State 命名空间，语义错误。
5. 恢复时三者必须回到同一个一致进度，否则可能重放了数据却没有对应 Timer，或恢复了 Timer 却缺少订单 State，造成漏告警/重复告警。
6. 超时告警本身已经是一个发生过的业务事件。晚到 PAY 可以产生“恢复/关闭告警”新事件，但不应该篡改历史事实。

27. 本讲最终验收清单

- 能说明 Window 与 State + Timer 的适用问题差异。
- 能说清 Event-Time Timer 与 Processing-Time Timer 的区别。
- orders_timeout_l8 创建成功且 3 个 Partition 正常。
- Flink Job 能 RUNNING，TaskManager 有输入记录。
- O3001 输出 PAID_IN_TIME 且不产生超时告警。
- O3002 产生且只产生一次 TIMEOUT_ALERT。
- O3003 输出 CANCELLED 且不产生超时告警。
- O3004 产生且只产生一次 TIMEOUT_ALERT。
- O3999 能推动 Watermark 并最终输出 PAID_IN_TIME。
- Checkpoint 页面有 Completed Checkpoints。
- 能解释为什么“没有后续 Event Time”时 Event-Time Timer 可能不触发。
- 能解释超时后再 PAY 为什么变成一个新的业务事实。

完成标准：你已经开始从“写流处理算子”进入“设计事件驱动状态机”。这比会几个 Flink API 更接近真实实时系统。